

코딩은 Claude와 함께, 배운 것은 내 것으로.

learn-notes는 Claude가 고친 코드가 무엇을 어떻게 달라지게 했는지 턴마다 학습 노트로 보여 주고, 잊을 때쯤 내가 만든 코드로 복습 퀴즈를 내 주는 Claude Code 모드입니다. 모드(MOD)가 무엇인지부터 설치, 기능, 활용 팁까지 한 번에 안내합니다.

버전 1.5.0

2026년 10월

github.com/Owen-YC/learn-notes

claude — ~/shop

> 결제된 주문만 세는 함수에 최소 금액 조건도 넣어줘

● Read(orders.js)
Read 12 lines

● Update(orders.js)
Updated orders.js with 4 additions and 4 removals

● 반복문을 for...of로 바꾸고 minAmount 인자(기본값 0)를 추가해 그 금액 이상인 결제 주문만 셉니다.

노트 2/2 · 05:12 · 파일 1개 · +4 -4
p: ◀ 이전 n: 다음 ▶ w: 다시 쓰기
요청: 결제된 주문만 세는 함수에 최소 금액 조건도 넣어줘
노트 · v: 전/후 · 개념 모음 · q: 퀴즈

한 줄 요약

결제 주문 개수를 셀 때 최소 금액 조건을 추가하고, 더 간결한 반복문으로 개선했습니다.

무엇이 바뀌었나

- 전 결제 완료된(paid가 참) 모든 주문의 개수를 세었습니다.
+ 후 결제 완료되면서 일정 금액 이상인 주문만 셉니다.
→ 예 countPaid([{paid: true, amount: 50}, {paid: true, amount: 10}], 30) → 전: 2 / 후: 1

왜 이렇게 바꿨을까

사용자가 최소 금액 조건을 요청했으므로, minAmount 매개변수(함수가 받는 값)를 추가했습니다. 기본값 0으로 설정해서, 금액을 지정하지 않으면 이전처럼 모든 결제 주문을 세도록 했습니다. 동시에 구식 for 반복문 대신 for...of를 사용해 코드를 더 읽기 쉽고 안전하게 만들었으며, 변수 업데이트도 간단하게 표현했습니다.

배울 개념

- for...of 루프 (복습): 배열의 각 요소를 하나씩 꺼내며 순회하는 반복문입니다. - for (const order of orders)
- 기본 매개변수 (복습): 함수를 호출할 때 값을 주지 않으면 자동으로 사용될 기본값입니다. - function countPaid(orders, minAmount = 0)
- 복합 할당 연산자: +=는 "현재 값에 더하고 그 결과를 저장"하는 간단한 표기법입니다. n += 1은 n = n + 1과 같은 뜻입니다. - n += 1

직접 확인해 볼 것

- 다양한 minAmount 값(예: 0, 20, 30, 100)을 넣어서 결과가 어떻게 달라지는지 확인해보기
- minAmount를 '널지' 않고 호출했을 때 기본값 0이 제대로 적용되는지 확인해보기

새로 배운 개념 복합 할당 연산자

다시 만난 개념 for...of 루프 x2 · 기본 매개변수 x2

이 노트로 t: 퀴즈 e: 더 쉽게 r: 예시로 따라가기

i: 질문하기

질문 이 노트에서 궁금한 것을 적고 Enter (예: 왜 const를 썼...

들어가며

10분이면 다 읽고, 오늘부터 바로 쓸 수 있습니다.

이 안내서는 팀에서 Claude Code로 코딩하는 분들을 위해 썼습니다. 특히 코드를 직접 쓰기보다 Claude에게 맡기는 일이 많은 분, 코딩을 배우는 중인 분께 도움이 됩니다. 처음 듣는 말이 나와도 괜찮습니다. 필요한 용어는 그때그때 풀어 설명합니다.

1

모드(MOD)란 무엇인지

Claude Code에 기능을 더하는 확장이 어떻게 동작하는지 이해합니다.

2

설치하고 첫 노트 보기

두 줄로 설치하고, 첫 요청에서 학습 노트를 받아 봅니다.

3

배운 것을 내 것으로

질문, 퀴즈, 간격 반복 복습, 기록으로 실력을 쌓습니다.

차례

- | | | |
|----|----------------------|-------------------------|
| 1 | 왜 learn-notes인가요? | 바이브코딩의 장점과 함정 |
| 2 | Claude Code와 모드(MOD) | 개념 · 용어 · 동작 원리 |
| 3 | 설치하고 첫 노트 보기 | 5단계 · Windows 팁 |
| 4 | 학습 노트 읽기 | 노트 다섯 칸 · 전/후 한눈에 |
| 5 | 노트에서 바로 묻기 | 퀴즈 · 더 쉽게 · 따라가기 · 질문하기 |
| 6 | 내 코드로 푸는 복습 퀴즈 | 힌트 · 채점 · 간격 반복 |
| 7 | 쌓이는 기록 | 개념 모음 · 정리 · 통계 · Anki |
| 8 | 명령 한눈에 보기 | /learn 하위 명령 |
| 9 | 설정 · 비용 · 개인정보 | 무엇이 어디로 가는지 |
| 10 | 문제 해결 | 노트가 안 생길 때 |
| 11 | 팀에서 이렇게 써 보세요 | 하루 한 주 루틴 |

1 왜 learn-notes인가요?

코딩 속도는 그대로, 배우는 것은 덤으로.

Claude에게 "결제된 주문만 세는 함수에 최소 금액 조건도 넣어 줘"라고 말하면 몇 초 만에 코드가 바뀝니다. 이렇게 AI에게 코드를 맡기며 만드는 방식을 흔히 **바이브코딩**이라고 부릅니다. 빠르고 편하지만, 하루가 끝나고 돌아보면 "무엇이 왜 바뀌었는지"는 머리에 거의 남지 않습니다.



왜 바뀌었는지 모릅니다

Claude의 설명은 대화 속에 묻히고, 바뀐 코드의 전후를 다시 찾아보기 어렵습니다.



같은 개념을 매번 처음 봅니다

지난주에 본 `for...of` 를 오늘 또 처음 보는 것처럼 지나칩니다.



배운 것이 남지 않습니다

무엇을 배웠는지 기록이 없어, 성장했는지 스스로 알기 어렵습니다.

learn-notes는 이 세 가지를 Claude Code 안에서 바로 해결합니다. 따로 공부 시간을 내지 않아도, 평소처럼 코딩을 요청하기만 하면 됩니다.



언마다 학습 노트

Claude가 파일을 고친 턴이 끝나면, 무엇이 왜 바뀌었고 무엇을 배울 수 있는지 노트로 정리합니다.



잊을 때쯤 복습 퀴즈

배운 개념을 모아 두었다가, 잊을 때쯤 내가 만든 코드로 문제를 냅니다. 답은 Claude가 채점합니다.



쌓이는 기록

날짜별 일지, 개념 모음, 연속 학습일과 정답률이 쌓여 성장이 눈에 보입니다.

"Claude가 코드를 고칠 때마다, 나에게는 짧은 수업이 하나씩 남습니다."

claude -- ~/shop

전체 화면 · 200x50 · 패널: ctrl+x tab으로 고른 뒤 v 보기 · q 퀴즈 · i 입력

> 결제된 주문만 세는 함수에 최소 금액 조건도 넣어줘

- Read(orders.js)
└ Read 12 lines
- Update(orders.js)
└ Updated orders.js with 4 additions and 4 removals

* Responding...

• 작업 중: 파일 1개 · 턴이 끝나면 노트를 씁니다
orders.js +4 -4

노트 1/1 · 지난 세션 · 10-02 06:12 · 파일 1개 · +5 -5
p: ◀ 이전 n: 다음 ▶ w: 다시 쓰기
요청: cart.js의 total 함수를 부르면 에러가 나. 고쳐주고, 할인...
노트 · v: 전/후 · 개념 모음 · q: 퀴즈

한 줄 요약
배열 범위 초과 버그를 수정하고 할인을 매개변수를 추가했습니다.

무엇이 바뀌었나
cart.js
- 전 i <= items.length 조건으로 배열의 범위를 초과해서 에러가 발생했습니다.
+ 후 for...of 루프로 올바르게 각 항목을 순회하고, 할인율을 받아 적용한 합계를 반환합니다.
→ 예 total([{price: 100, qty: 2}]) → 전: 예러 / 후: 200, total([...], 0.1) → 후: 180 (10% 할인 적용)

왜 이렇게 바꿨을까

왼쪽은 평소의 Claude Code 대화, 오른쪽이 learn-notes 패널입니다. Claude가 일하는 동안 바뀌는 파일이 패널 맨 위에 실시간으로 쌓입니다.

2 Claude Code와 모드(MOD)

모드는 Claude Code에 끼우는 작은 확장입니다.

Claude Code는

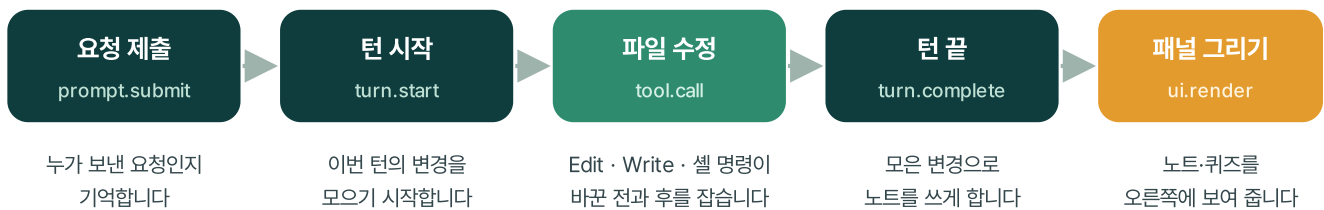
Anthropic이 만든 AI 코딩 도우미입니다. 터미널(또는 데스크톱 앱)에서 Claude와 대화하면, Claude가 파일을 읽고 코드를 고치고 명령을 실행합니다. 내가 요청을 보내고 Claude가 일을 마치고 답하기까지를 **턴(turn)** 하나라고 부릅니다.

모드(MOD)는

스마트폰에 앱을 설치하면 새 기능이 생기듯, **모드를 설치하면 Claude Code에 새 화면과 동작이 생깁니다.** 게임의 "모드"처럼 원래 프로그램은 그대로 두고 그 위에 기능을 엮는다고 생각하면 쉽습니다. Claude가 일하는 방식은 바뀌지 않고, 모드는 옆에서 지켜보다가 필요한 것만 채깁니다.

모드는 이렇게 동작합니다

Claude Code의 한 턴은 몇 가지 이벤트로 이어집니다. learn-notes의 **훅**이 각 이벤트에서 하는 일은 아래와 같습니다.



턴이 끝나면

작은 모델(기본 haiku)이 바뀐 코드와 요청을 읽고 노트를 씁니다 → 패널에 표시 · 일지 파일에 저장 · 배운 개념을 개념 모음에 더합니다

알아 두면 좋은 용어

용어	뜻	learn-notes에서는
마켓플레이스	모드(플러그인)를 나눠 주는 목록입니다. 보통 GitHub 저장소 하나입니다.	<code>Owen-YC/learn-notes</code>
플러그인	설치하고 지우는 단위입니다. 설정 항목, 명령, 훅을 한데 담은 꾸러미입니다.	<code>learn-notes@learn-notes</code>
모드 (mod)	플러그인 안에서 훅으로 Claude Code의 화면과 동작을 바꾸는 코드입니다.	노트·패널·퀴즈를 만드는 코드 전체
이벤트	Claude Code 안에서 일어나는 일입니다. 요청 제출, 턴 시작, 파일 수정, 턴 끝, 화면 그리기 등.	파일이 바뀌는 순간을 알아챍니다
훅 (hook)	특정 이벤트가 일어날 때 끼어들어 실행되는 함수입니다.	바뀌기 전과 후를 잡아 둡니다

모드가 할 수 있는 것, learn-notes가 쓰는 곳



패널

대화 옆 전용 화면. 노트·전/후·개념
모음·퀴즈를 보여 줍니다.



슬래시 명령

`/learn` 하나로 퀴즈, 정리, 질문,
통계를 엽니다.



상태줄·알림

복습할 개념 수를 상태줄에, 노트
완성을 알림으로 알려 줍니다.



작은 모델 호출

노트·퀴즈·채점·질문의 답을 저렴한
모델(haiku)이 씁니다.



파일 저장

날짜별 마크다운 일지와 개념 모음을
내 컴퓨터에 씁니다.



세션을 넘는 저장소

Claude Code를 다시 켜도 지난
노트와 퀴즈가 이어집니다.

안심하세요

모드는 내 컴퓨터의 Claude Code 안에서 돌고, Claude Code가 허락한 기능만 쓸 수 있습니다. learn-notes가
만드는 노트와 기록은 모두 내 컴퓨터에 저장되고, 모델 호출은 지금 쓰는 Claude Code 계정으로 나갑니다.

3

설치하고 첫 노트 보기

준비물: Claude Code v2.1.287 이상 (claude --version으로 확인)

1

설치합니다

Claude Code 안에서 두 줄을 입력합니다. 앞 줄은 learn-notes를 나눠 주는 곳(마켓플레이스)을 등록하고, 뒤 줄은
설치합니다.

```
/plugin marketplace add Owen-YC/learn-notes  
/plugin install learn-notes@learn-notes
```

2

다시 시작하고 전체 화면으로 바꿉니다

Claude Code를 껐다 켜 뒤 `/tui fullscreen` 을 입력합니다. 패널이 대화 오른쪽에 붙으려면 전체 화면이어야
합니다. 전체 화면이 아니면 프롬프트 위에 열립니다.

3

패널을 엽니다

`/learn` 을 입력합니다. 아직 노트가 없다는 안내와 첫 요청 예시가 보이면 성공입니다.

4

평소처럼 코딩을 요청합니다

예: `hello.js`에 이름을 받아 인사하는 함수를 만들어 줘. Claude가 파일을 만들거나 고친 턴이 끝나면 몇 초 안에 노트가 뜹니다.

5

단축키는 패널을 고른 뒤 누릅니다

`ctrl` + `x` 다음 `tab`으로 패널을 고르면(또는 마우스로 클릭하면) 버튼 앞에 적힌 키가 먹습니다. 예: `v` 보기 바꾸기, `q` 퀴즈.

WINDOWS 사용자

PowerShell에서 `irm https://claude.ai/install.ps1 | iex`로 Claude Code를 설치한 뒤 위와 똑같이 씁니다. Claude Code는 **작업할 폴더에서** 켜세요(`cd ~\practice` 뒤 `claude`). 관리자 PowerShell처럼 `C:\Windows\System32`에서 켜면 파일이 임시 폴더에 만들어집니다. 그래도 노트는 생기고, 패널에 안내가 뜹니다.

업데이트 · 지우기

업데이트: `/plugin` → `Installed` → `learn-notes` → `Update`, 그다음 `/reload-plugins`.

지우기: 터미널에서 `claude plugin uninstall learn-notes@learn-notes`. 일지 파일 (`~/.claude/learning-notes/`)은 남습니다.

클라우드 세션(웹·앱)에서는 패널 대신 명령으로 씁니다.

학습 노트 읽기

노트는 다섯 칸으로, 350단어 안쪽으로 짧게 씁니다.

칸	무엇이 담기나
한 줄 요약	이번 턴에 무엇을 했는지 한 문장으로
무엇이 바뀌었나	코드가 하던 일(- 전), 이제 하는 일(+ 후), 차이가 드러나는 입력과 결과(→ 예)를 한 줄씩
왜 이렇게 바꿨을까	요청과 Claude의 설명, 바뀐 코드를 근거로 이유를 풀어 줍니다
배울 개념	1~3개. 개념 이름, 쉬운 설명, 그 개념이 쓰인 코드 한 줄
직접 확인해 볼 것	직접 실행하거나 바꿔 보며 확인할 것 1~2개

노트 아래에는 이 노트에서 **새로 배운 개념**과 전에 배운 적 있는 **다시 만난 개념**(만난 횟수와 함께)이 나뉘어 나옵니다. 이미 배운 개념은 같은 이름으로 쌓이고, 노트에 "(복습)"이 붙습니다.

설명 수준

설정의 level로 노트의 눈높이를 고릅니다. **beginner**(기본)는 용어를 괄호 안에 쉬운 말로 풀고 비유를 곁들이고, **intermediate**는 개념과 "왜"에, **advanced**는 설계와 트레이드오프에 무게를 둡니다.

노트 2/2 · 05:12 · 파일 1개 · +4 -4
p: ◀ 이전 n: 다음 ▶ w: 다시 쓰기
요청: 결제된 주문만 세는 함수에 최소 금액 조건도 넣어줘
노트 · v: 전/후 · 개념 모음 · q: 퀴즈

한 줄 요약

결제 주문 개수를 셀 때 최소 금액 조건을 추가하고, 더 간결한 반복문으로 개선했습니다.

무엇이 바뀌었나

- 전 결제 완료된(paid가 참) 모든 주문의 개수를 세었습니다.
+ 후 결제 완료되면서 일정 금액 이상인 주문만 셉니다.
→ 예 countPaid([{"paid": true, amount: 50}, {"paid": true, amount: 10}], 30) → 전: 2 / 후: 1

왜 이렇게 바꿨을까

사용자가 최소 금액 조건을 요청했으므로, minAmount 매개변수(함수가 받는 값)를 추가했습니다. 기본값 0으로 설정해서, 금액을 지정하지 않으면 이전처럼 모든 결제 주문을 세도록 했습니다. 동시에 구식 for 반복문 대신 for...of를 사용해 코드를 더 읽기 쉽고 안전하게 만들었으며, 변수 업데이트도 간단하게 표현했습니다.

배울 개념

- **for...of 루프 (복습)**: 배열의 각 요소를 하나씩 꺼내며 순회하는 반복문입니다. - for (const order of orders)
- **기본 매개변수 (복습)**: 함수를 호출할 때 값을 주지 않으면 자동으로 사용될 기본값입니다. - function countPaid(orders, minAmount = 0)
- **복합 할당 연산자: +=**는 "현재 값에 더하고 그 결과를 저장"하는 간단한 표기법입니다. n += 1은 n = n + 1과 같은 뜻입니다. - n += 1

직접 확인해 볼 것

1. 다양한 minAmount 값(예: 0, 20, 30, 100)을 넣어서 결과가 어떻게 달라지는지 확인해보기
2. minAmount를 넣지 않고 호출했을 때 기본값 0이 제대로 적용되는지 확인해보기

새로 배운 개념 복합 할당 연산자

다시 만난 개념 for...of 루프 x2 · 기본 매개변수 x2

이 노트로 t: 퀴즈 e: 더 쉽게 r: 예시로 따라가기
i: 질문하기

질문 이 노트에서 궁금한 것을 적고 Enter (예: 왜 const를 썼...)

실제 노트(haiku가 쓴 글)

보기 바꾸기 패널 위쪽 줄

패널 위쪽 줄에 **노트 · 전/후 · 개념 모음 · 퀴즈**가 있고, 지금 보는 것은 굵게 밑줄이 쳐집니다. 이름을 클릭하면 바로 그 보기로 갑니다.

키	하는 일
v	다음 보기로: 노트 → 전/후 → 개념 모음 → 노트. 퀴즈에서는 노트로 돌아갑니다
q	어디서든 퀴즈로
p n	이전 · 다음 노트. 지난 세션의 노트까지 거슬러 올라갑니다
w	노트를 다시 씁니다. 다시 쓰다 실패하면 원래 노트는 그대로 둡니다

```
claude - ~/shop

> 결제된 주문만 세는 함수에 최소 금액 조건도 넣어줘

● Read(orders.js)
  └ Read 12 lines

● Update(orders.js)
  └ Updated orders.js with 4 additions and 4 removals

● 반복문을 for...of로 바꾸고 minAmount 인자(기본값 0)를 추가해 그 금액 이상인 결제 주문만 셉니다.

노트 2/2 · 05:12 · 파일 1개 · +4 -4
p: ◀ 이전 n: 다음 ▶ w: 다시 쓰기
요청: 결제된 주문만 세는 함수에 최소 금액 조건도 넣어줘
노트 · 전/후 · v: 개념 모음 · q: 퀴즈

- 전 결제 완료된(paid가 참) 모든 주문의 개수를 세었습니다.
+ 후 결제 완료되면서 일정 금액 이상인 주문만 셉니다.
→ 예 countPaid([{{paid: true, amount: 50}, {paid: true, amount: 10}}, 30) → 전: 2 / 후: 1

바뀐 줄은 -·+로, 그 줄에서 바뀐 낱말은 굵게 표시합니다

orders.js
수정 · +4 -4
- 전 · 3~8행
3 // 결제 완료된 주문 수를 셉다
4 - function countPaid(orders) {
5 -   var n = 0
6 -   for (var i = 0; i < orders.length; i++) {
7 -     if (orders[i].paid) n = n + 1
8   }
```

전/후 보기. 맨 위에 노트의 전·후·예가 다시 나오고, 바뀐 곳마다 원래 코드(전)와 바뀐 코드(후)를 줄 번호 그대로 놓습니다. 바뀌지 않은 줄은 흐리게, 바뀐 줄은 -·+로, 그 줄에서 바뀐 낱말만 굵은 색으로 칠합니다(var → let처럼).

5 노트에서 바로 묻기

노트 맨 아래 "이 노트로" 줄에서 이어 갑니다.

t

이 노트 퀴즈

방금 읽은 노트의 개념만으로 문제를 받습니다. 읽자마자 확인하기 좋습니다.

e

더 쉽게

문장을 짧게, 용어를 일상어로, 개념마다 일상의 비유를 넣어 다시 씁니다. **w** 로 원래 수준으로 돌아갑니다.

r

예시로 따라가기

예시 입력 하나로 바뀐 코드를 한 단계씩 따라가며 변수 값이 어떻게 바뀌는지, 바뀌기 전 코드라면 어디서 결과가 갈리는지 보여 줍니다.

i

질문하기

노트 아래 질문칸에 적고 Enter. 그 노트와 코드를 근거로 짧게 답합니다.

질문칸에 "기본값 0은 왜 넣은 거야? 안 넣으면 어떻게 돼?"처럼 궁금한 것을 그대로 적으면 됩니다. 답은 노트 바로 아래에 남고(노트마다 최근 3개), 일지 파일에도 기록됩니다.

이어서 "그럼 음수를 넣으면?"처럼 물으면 **앞의 질문과 답을 읽고** 이어서 답합니다.

대화창의 Claude에게 묻는 것과 무엇이 다른가요?

노트 아래 질문은 코드를 고치지 않고, 그 노트와 바뀐 코드만 보고 저렴한 모델이 짧게 답합니다. 작업 흐름을 끊지 않고 궁금한 것만 바로 풀 수 있습니다. 명령으로는 `/learn ask` 질문입니다.

1. 다양한 `minAmount` 값(예: 0, 20, 30, 100)을 넣어서 결과가 어떻게 달라지는지 확인해보기
2. `minAmount`를 넣지 않고 호출했을 때 기본값 0이 제대로 적용되는지 확인해보기

새로 배운 개념 복한 할당 연산자

다시 만난 개념 `for...of` 루프 $\times 2$ · 기본 매개변수 $\times 2$

이 노트로 **t**: 퀴즈 **e**: 더 쉽게 **r**: 예시로 따라가기
i: 질문하기

질문 · 기본값 0은 왜 넣은 거야? 안 넣으면 어떻게 돼?

기본값 0을 넣은 이유는 노트에 설명되어 있듯이, 함수를 호출할 때 금액을 지정하지 않으면 자동으로 0이 사용돼서 이전처럼 모든 결제 주문을 세도록 하기 위함입니다. 비유하자면 매점에 가서 "할인을 말하지 않으면 정가로 줄게"라고 미리 정해놓는 것입니다.

안 넣으면 어떻게 될까요? 일반적으로 말하자면, `minAmount`를 항상 입력해야 합니다. 입력하지 않으면 값이 정해지지 않은 상태(`undefined`)가 돼서 비교가 제대로 작동하지 않습니다.

```
// 기본값이 있을 때
countPaid(orders) // ✓ minAmount = 0 으로 자동 설정
countPaid(orders, 30) // ✓ minAmount = 30 으로 설정

// 기본값이 없을 때
countPaid(orders) // ✗ minAmount가 정해지지 않음 → 오
countPaid(orders, 30) // ✓ minAmount = 30
```

질문 이 노트에서 궁금한 것을 적고 Enter (예: 왜 `const`를 썼...

노트 아래에 남은 질문과 답(haiku가 실제로 쓴 글)

6

내 코드로 푸는 복습 퀴즈

잇을 때쯤, 내가 만든 코드로 다시 떠올립니다.

q로 퀴즈를 열고 **s**로 문제를 받습니다(최대 3문제). **복습할 때가 된 개념부터**, 그 개념을 처음 배운 노트의 **내 코드**를 보여 주며 묻습니다. 일반적인 예제가 아니라 내가 Claude와 함께 만든 코드라서 훨씬 잘 떠오릅니다.

키	하는 일
s	문제 받기 · 새 문제 받기
i	답 적기. 입력칸에 적고 Enter 를 누르면 Claude가 채점합니다
h	힌트. 답을 말하지 않고 떠올릴 실마리만 보여 줍니다
a	정답만 보기. 그다음 o 맞혔어요 · x 틀렸어요로 스스로 채점
f	채점 바꾸기. Claude의 채점이 이상하면 맞힘 ↔ 틀림

Claude는 **✓ 맞힘** · **△ 거의 맞음** · **✕ 틀림** 중 하나로 채점하고, 내 답에서 맞은 점과 빠진 점을 한두 문장으로 짚어 줍니다. 표현이 달라도

v: 노트 · 전/후 · 개념 모음 · 퀴즈

복습 퀴즈 · 05:12 · 3문제 중 0개 채점

s: 새 문제 받기

문제 1/3

다음 코드에서 `for` 루프의 조건을 잘못 쓰면 배열 범위를 넘어설 수 있어요. 올바른 조건은 될까요?

```
for (let i = 0; i ____ ; i++) {
  sum += items[i].price * items[i].qty
}
```

힌트

`items.length`는 배열의 총 개수인데, 인덱스는 0부터 시작하므로 마지막 위치는 `length-1`입니다.

내 답 답을 적고 Enter: Claude가 채점

i: 답 적기 a: 정답만 보기

장바구니 코드로 낸 문제와 힌트(h)

뜻이 같으면 맞습니다.

간격 반복 복습 잊을 때쯤 다시

개념마다 복습 단계가 있습니다. 때가 된 개념을 맞힐 때마다 다음 복습까지의 간격이 늘어납니다.

1일 0단계	3일 1단계	7일 2단계	14일 3단계	30일 4단계	60일 5단계
 맞이면 한 단계 올라 더 나중에 나옵니다. 때가 되기 전이나 같은 날 다시 맞힌 것은 단계를 그대로 둡니다.	 거의 맞으면 한 단계 내려가 조금 더 빨리 다시 나옵니다.	 틀리면 처음 단계로 돌아가 다음 퀴즈 맨 앞에 나옵니다.			

claude -- ~/shop

전체 화면 · 200x50 · 패널: ctrl+x tab으로 고른 뒤 v 보기 · q 퀴즈 · i 입력

> 결제된 주문만 세는 함수에 최소 금액 조건도 넣어줘

- Read(orders.js)
└ Read 12 lines
- Update(orders.js)
└ Updated orders.js with 4 additions and 4 removals
- 반복문을 for...of로 바꾸고 minAmount 인자(기본값 0)를 추가해 그 금액 이상인 결제 주문만 셉니다.

v: 노트 · 전/후 · 개념 · 모음 · 퀴즈

복습 퀴즈 · 05:12 · 3문제 중 1개 채점

s: 새 문제 보기

x 틀림 1. 배열 경계 오류 · 방금 채점

아쉽지만 틀렸습니다. 루프 안에서 sum을 더하는 부분을 설명하셨는데, 문제는 i ____ 빈칸에 들어갈 루프 조건을 묻는 것입니다. 배열의 길이와 인덱스를 비교하는 조건식이 필요합니다.

내 답: sum에 더하는 거요

정답

i < items.length가 올바릅니다. i <= items.length로 쓰면 범위를 초과해 undefined를 만나거나 에러가 발생합니다.

f: 채점 바꾸기 (맞힘으로)

문제 2/3

다음 코드에서 n += 1을 긴 형태로 풀어서 쓴다면 어떻게 될까요?

if (order.paid && order.amount >= minAmount) n += 1

내 답 답을 적고 Enter: Claude가 채점

i: 답 적기 h: 힌트 a: 정답만 보기

"sum에 더하는 거요"라는 답에 Claude가 △ 거의 맞음과 빠진 점("더한 결과를 다시 저장한다")을 짚었습니다. 채점이 이상하면 **f** 로 바꿉니다. 복습 단계도 처음부터 그렇게 채점한 것처럼 돌아갑니다.

상태줄 알림

복습할 개념이 있으면 프롬프트 아래 상태줄에 학습 노트 · 복습할 개념 3개 · /learn 패널에서 q가 뜹니다. 끄려면 설정의 reviewReminder를 끕니다.

7

쌓이는 기록

무엇을 배웠는지, 얼마나 꾸준했는지 눈에 보입니다.

개념 모음

노트의 "배울 개념"을 프로젝트를 넘어 한 목록에 모읍니다. 개념마다 만난 횟수, 최근에 만난 날, 다음 복습 날짜, 최근 설명, 그 개념이 나온 노트(누르면 그 노트로)가 있습니다. 맨 위에는 연속 학습일과 최근 7일 진도, 지금 복습할 개념이 나옵니다.

표기만 다른 이름("구조 분해 할당 (Destructuring)"과 "구조 분해 할당")은 한 개념으로 셉니다. 그래도 갈라졌다면 `/learn merge A = B` 로 합치고, 거꾸로 하면 되돌립니다.

하루 · 한 주 정리

`/learn recap` 은 오늘 노트를 모두 읽고 **한 일 · 핵심 개념 ·** **헛갈리기 쉬운 것 · 다음에 해 볼 것**으로 정리합니다. `어제` · `이번주` · `최근 7일` · 날짜도 됩니다. 정리는 일지에 남습니다.

v: 노트 · 전/후 · **개념 모음** · q: 퀴즈

배운 개념 4개 · 최근 7일 새로 4 · 다시 만날 2
연속 2일째 · 최근 7일 노트 2개

복습할 개념 1개 · 틀린 것 먼저, 잊을 때쯤 다시 나옵니다 · q로 퀴즈
배열 경계 오류 · 오늘

모든 개념 · 여러 번 만난 순

for...of 루프 x2 · 최근 05:12 · 다음 복습 3일 뒤
배열의 각 요소를 하나씩 꺼내며 순회하는 반복문입니다. - for (const order of orders)
노트: 10-02 06:12 cart.js 05:12 orders.js

기본 매개변수 x2 · 최근 05:12 · 다음 복습 3일 뒤
함수를 호출할 때 값을 주지 않으면 자동으로 사용될 기본값입니다. - function countPaid(orders, minAmount = 0)
노트: 10-02 06:12 cart.js 05:12 orders.js

복합 할당 연산자 x1 · 최근 05:12 · 다음 복습 내일
+=는 "현재 값에 더하고 그 결과를 저장"하는 간단한 표기법입니다. n += 1은 n = n + 1과 같은 뜻입니다. - n += 1
노트: 05:12 orders.js

배열 경계 오류 x1 · 최근 10-02 06:12 · 다음 복습 오늘
배열의 유효한 인덱스는 0부터 length-1까지입니다. 실수로 범위를 초과하면 undefined를 만나거나 에러가 발생합니다. - for (var i = 0; i <= items.length; i++) ← 잘못된 조건
노트: 10-02 06:12 cart.js

개념 모음 보기

학습 통계와 Anki

/learn stats

연속 학습일(가장 길었던 날 수), 최근 7일 노트·새 개념·퀴즈, 최근 30일 정답률, 날짜별 막대를 보여 줍니다. 모든 프로젝트를 합쳐 최근 120일을 남깁니다.

/learn anki

지금까지 낸 퀴즈 문제(최근 300개)와 개념을 Anki 카드 파일로 씁니다. Anki 데스크톱에서 파일 → 가져오기로 고르면 `learn-notes` 데크에 들어가고, 휴대폰 앱과 동기화됩니다.

파일은 어디에 있나요

무엇	어디에
날짜별 일지	<code>~/.claude/learning-notes/2026-10-03_shop_xxxxx.md</code> · 노트, 바뀐 코드, 질문과 답, 정리
개념 모음 표	같은 폴더의 <code>concepts.md</code>
Anki 카드	같은 폴더의 <code>learn-notes-anki.txt</code>
지난 노트 · 퀴즈 · 기록	Claude Code의 플러그인 저장소(프로젝트마다 최근 노트 30개)

명령 한눈에 보기

한글로도 됩니다: /learn 퀴즈 · 개념 · 정리 · 기록 · 질문 · 찾기 · 일지 · 도움말

명령	하는 일
<code>/learn</code>	학습 노트 패널 열기 (패널을 띄울 수 없는 화면이면 마지막 노트를 대화창에)
<code>/learn quiz</code>	복습할 개념으로 문제 받기 (내 코드로 묻습니다)
<code>/learn quiz 1 내 답</code>	1번 문제에 답을 적어 Claude에게 채점받기. 다음에 할 일을 알려 줍니다
<code>/learn quiz 힌트</code>	아직 안 푼 문제들의 힌트
<code>/learn quiz 정답</code>	정답 보기 (채점은 하지 않습니다)
<code>/learn quiz 맞음 1 3</code>	정답을 본 뒤 스스로 채점. 틀림 2 도 됩니다. 이미 채점한 문제도 바꿉니다
<code>/learn concepts</code>	지금까지 배운 개념과 다음 복습 날짜
<code>/learn recap</code>	오늘 배운 것 정리 (어제 · 이번주 · 최근 7일 · 날짜)
<code>/learn stats</code>	연속 학습일, 최근 7일, 30일 정답률
<code>/learn ask 질문</code>	패널에서 고른 노트(없으면 마지막 노트)에 대해 묻기
<code>/learn last</code>	마지막 노트를 대화창에
<code>/learn find 말</code>	모든 프로젝트의 노트에서 요청·내용·파일 이름·개념으로 찾기
<code>/learn day</code>	오늘 노트 목차 (어제 · 날짜). <code>/learn days</code> 는 일지가 있는 날짜
<code>/learn anki</code>	퀴즈 문제와 개념을 Anki 카드 파일로
<code>/learn merge A = B</code>	이름만 다른 같은 개념 합치기 (거꾸로 하면 되돌림)
<code>/learn save</code>	아직 파일에 없는 노트 저장 (자동 저장을 껐을 때)
<code>/learn clear</code>	이 프로젝트의 노트 비우기 (일지와 개념 모음은 그대로)
<code>/learn help</code>	명령 목록

패널이 안 보이는 곳에서도

클라우드 세션이나 모바일 앱처럼 패널이 안 보이는 곳에서는 명령만으로 다 할 수 있습니다. `/learn quiz`로 문제를 받고, `/learn quiz 1 내 답`으로 채점받고, `/learn ask`로 묻습니다.

항목	기본값	설명
<code>autoNote</code>	켜짐	끄면 바뀐 코드만 모으고, 노트는 패널에서 <code>w</code> 를 눌렀을 때만 씁니다 (모델 비용 0)
<code>model</code>	haiku	haiku · sonnet · opus. 노트·퀴즈·채점·질문에 씁니다. haiku가 가장 싸고 빠릅니다
<code>level</code>	beginner	beginner(용어를 쉬운 말로) · intermediate(개념 위주) · advanced(설계·트레이드오프)
<code>autoOpen</code>	켜짐	첫 노트가 생기면 패널을 저절로 엽니다 (전체 화면, 144칸 이상일 때만)
<code>autoSave</code>	켜짐	노트를 날짜별 마크다운 일지로 저장합니다. 정리(recap)도 이 일지를 읽습니다
<code>reviewReminder</code>	켜짐	복습할 개념이 있으면 상태줄에 개수를 띄웁니다
<code>saveDir</code>	(비움)	비우면 <code>~/.claude/learning-notes</code>

비용

파일을 바꾼 때마다 모델 호출이 **한 번** 일어납니다(기본 haiku). 퀴즈 받기 · 답 채점 · 질문 · 따라가기 · 더 쉽게 · 정리는 할 때마다 한 번씩입니다. 통계 · Anki · 개념 모음은 모델을 부르지 않습니다.

개인정보

바뀐 코드와 요청 문장이 모델 호출에 실려, 지금 쓰는 Claude Code 계정으로 나갑니다. 노트 · 일지 · 기록은 내 컴퓨터에만 저장됩니다. 비밀값이 든 파일을 다룬다면 `autoNote`를 끄세요.

모으지 않는 것

Claude가 스스로 쓰는 계획·메모리 파일, 프로젝트 밖 임시 폴더의 작업 파일, `git stash·checkout·pull`처럼 디스크에 되돌리거나 가져온 내용은 노트에 담지 않습니다. 내가 배울 코드가 아니기 때문입니다.

문제 해결

노트가 안 생기면 위에서부터 하나씩 확인해 보세요.

- ☐ **그 턴에 파일이 바뀌었나요?** 질문에 답만 한 턴은 노트가 없습니다. Claude가 파일을 만들거나 고친 턴에만 생깁니다.
- ☐ **패널이 열려 있나요?** `/learn`으로 엽니다. 대화 오른쪽에 붙으려면 `/tui fullscreen` (전체 화면)이어야 합니다. 아니면 프롬프트 위에 열리고, 노트가 준비되면 알림으로 알려 줍니다.
- ☐ **단축키가 안 먹나요?** `ctrl` + `x` `tab`으로 패널을 먼저 고르세요. 마우스로 클릭해도 됩니다.
- ☐ **/learn을 모른다고 하나요?** 다른 폴더에서 설치했을 수 있습니다. `/plugin`의 Installed 탭에서 learn-notes를 확인하고, 없으면 설치 두 줄을 다시 입력합니다.

- ☐ **노트 칸이 빨간 글씨인가요?** 이유가 적혀 있습니다. "서버가 붐빉니다" · "요청 한도에 걸렸습니다"는 잠시 뒤 **w**로 다시 쓰고, "모델을 부를 수 없습니다"는 `/config`에서 다른 모델을, "로그인이 필요합니다"는 `/login`.
- ☐ **Windows에서 파일이 엉뚱한 곳에 생기나요?** Claude Code를 작업할 폴더에서 커세요(`cd ~\practice` 뒤 `claude`).
- ☐ **업데이트가 안 보이나요?** `/plugin`에서 Update 뒤 `/reload-plugins`. `/learn help`에 "예시로 따라가기"가 보이면 1.5.0입니다.

그래도 안 되면

화면을 캡처해 github.com/Owen-YC/learn-notes/issues에 남기거나 팀 채널에 공유해 주세요. Claude Code 버전(`claude --version`)과 운영체제를 함께 적어 주시면 빨리 찾을 수 있습니다.

11

팀에서 이렇게 써 보세요

하루 5분, 한 주 15분이면 충분합니다.



아침: 복습 3문제

상태줄에 "복습할 개념"이 보이면 **q** → **s**로 3문제.
답을 적어 채점받으면 간격이 알아서 늘어납니다.



작업 중: 모르면 바로 질문

노트에서 낯선 개념이 보이면 **i**로 질문하고, 코드가 어떻게 도는지 궁금하면 **r**로 따라가 보고, 어려우면 **e**로 더 쉽게. 작업 흐름을 끊지 않습니다.



하루 끝: 오늘 정리

`/learn recap`으로 오늘 배운 것을 네 칸으로 정리합니다. "핵심 개념" 한 줄을 팀 채널에 공유해 보세요.



주 1회: 회고

`/learn recap` 이번주 와 `/learn stats`로 한 주를 돌아봅니다. 팀원끼리 "이번 주 다시 만난 개념"을 나누면서 배웁니다.

눈높이 맞추기

처음에는 설정 level을 **beginner**로 두고, 노트가 너무 쉽게 느껴지면 **intermediate**로 올리세요. 출퇴근길에는 `/learn anki`로 만든 카드로 복습할 수 있습니다.

빠른 참조 · 책상 옆에 두고 보세요

오늘 첫 요청부터 노트가 쌓입니다.

설치 (Claude Code 안에서)

```
/plugin marketplace add Owen-YC/learn-notes  
/plugin install learn-notes@learn-notes
```

다시 시작 → `/tui fullscreen` → `/learn` → 평소처럼 코딩 요청

패널 (ctrl+x tab으로 고른 뒤)

- v** 보기 바꾸기
- q** 퀴즈
- p** **n** 이전 · 다음 노트
- w** 다시 쓰기
- t** 이 노트 퀴즈
- e** 더 쉽게
- r** 예시로 따라가기
- i** 질문하기

퀴즈 보기

- s** 문제 받기
- i** 답 적기 → Enter로 채점
- h** 힌트
- a** 정답만 보기
- o** **x** 맞혔어요 · 틀렸어요
- f** 채점 바꾸기

자주 쓰는 명령

`/learn quiz`

복습 문제 받기

`/learn recap`

오늘 배운 것 정리

`/learn stats`

연속 학습일 · 정답률

`/learn quiz 1 내 답`

Claude가 채점

`/learn ask` 질문

노트에 대해 묻기

`/learn help`

전체 명령